OOP through Python

Q.) what are the main component of an application?

Ans:- **1. data**

2. codes that acts upon data

Q.) what are the programming methodologies for application?

Ans:- **1. POP :- procedural oriented programming**

2. OOP :- object oriented programming

Q.) what is POP?

Ans:- POP is nothing but style of application development (software development) in which application modularised (divided) using procedure.

Q.) what is OOP?

Ans:-OOP is one style of application development in which application modularized using objects.

Q.) what is the limitation of POP?

Ans:- data and code that acts upon data are not properly organized in POP style of application development.

Such lack of organization of data and code leads to following problems.

1. **Data is not secure (from whom ?).**
2. **Complexity can't be handle properly.**
3. **Real world entities can't be moulded properly**

Q.) how are the limitation of POP overcome?

Ans:- using OOP

Q.) what are the features of OOP?

1. encapsulation

2. polymorphism

3. inheritance

Q.) what is encapsulation?

Ans:- combining data and eligible code that acts upon data into a single unit of organization is nothing but encapsulation.

Wrapping up of data and code is encapsulation.

Q.) how to implement in encapsulation in python application?

Ans:- using class and object.

25/06/19

Q.) what is a class?

- **A class is a user defined data type.**
- **Programmatically realization of theoretical object oriented.**
- **Encapsulation is nothing but a class programmatic way means of implementing encapsulation is nothing but a class.**
- **A class is a blue print of an object that is a proposal object is nothing but a class.**
- **A class is an basics of implementing encapsulation in object oriented.**

Q.) what is an object?

- **Instance of a class is nothing but object.**
- **Fundamental unit of data storage in an object oriented system is nothing but an object.**
- **Object is an encapsulated unit.**
- **Object is software bundle that stores data and code acts upon that data.**

26/06/19

Q.) what is an state of an object?

Ans:- data stored in an object is known as the state of the object.

Q.) what is the behaviour of the object?

- **Method of an object is nothing but the behaviour of an object.**
- **Behaviour of an object only can change the state of an object.**

- That is method of that object only can process the data stored in the object this is the complete essence of encapsulation.

Q.) what is an identity object?

Ans:- name of an object is nothing but its identity.

Q.) what are the properties of an object?

- Variable (instance of variable) of the object are nothing but its property.
- Property of an object are also known as field, attributes or instance of variable.

Note:- property of an object from the structure of the object.

Q.) write a python program to represent 2 employee store employee data(empno, name, salary) and display their data:

```
#employee.py

class employee

    def storeEmployeeData(self):

        self.empno=10001

        self.name="rama"

        self.salary=50000

    def displayEmployeeDetails(self):

        print("empno:",self.empno)

        print("name:",self.name)

        print("salary:",self.salary)

e1=employee()

e1.storeEmployeeData()

print("first employee details")

e1.displayEmployeeDetails()

e2=employee()
```

```
e2.storeEmployeeData()

print("second employee details")

e1.displayEmployeeDetails()
```

27/06/19

Q.) what is the limitation of the above program?

- Any number of object can be created of a class. Objects of the same class have same structure and same behaviour, but practical reason sate of any objects may not be the same.
- In the above application both the employee objects have same state.

Reason:- for the same state of the two employee object is non parameterised method, storeEmployeeData()

Q.) how to overcome above limitation?

Ans:-make storeEmployeeData() method parameterised.

```
    def storeEmployeeData(self,empno,name,salary):

        self.empno=empno

        self.name=name

        self.salary=salary
```

while calling this method on the different objects, supply different arguments.

```
    e1.storeEmployeeData(10001,'rama',45000)

    e2.storeEmployeeData(10002,'ramu',25000)
```

Note:- here, storeEmployeeData() method is called on the employee object using the reference (with different argument).

Q.) explain about "self" parameter of the instance method?

- "self" is an implicit reference (automatic pointer) to the current object.
- At any given point in time , a method is called on which object is known as current object.

- **Python documentation from PSF (python software foundation) says, better have this name "self" but is an identifier. It can have any name.**
- **Whenever an instance method is called on an object, python interpreter implicitly supplies that object address into the variable "self".**

Q.)what is an instance variable?

- **A variable that is defined within a special method of a class using self is nothing but an instance variable.**
- **Instance variable are the properties (attributes) of an object.**
- **For each instance (object) of the class one separate copy of these variables is created and hence the name.**
- **(per)instance variable are used to represent individual object level information.**

For eg.

Empno,name,salary	**of each employee**
Accno,name,balace	**of each account**
Rollno,name,marks	**of each students**

- **Instance variable can be accessed in all instance method of the class using the reference variable "self".**

Q.) what is an instance method?

- **A method of a class that takes object reference (self) as the first argument is nothing but an instance method.**
- **An instance method can be called only on the instance(object) and hence the name.**
- **An instance method is the behaviour of the object.**
- **A instance method with the state (data stored in the instance variable) of the object.**
- **An instance method is the "interface" between an object and the outside world.(other parts of the same application).**

Q.) what happens if fisplyEmployeeDetails() method is called on the employee object before calling storeEmployeeData() method in "employee.py"?

Ans:-AttributError: "Employee" object has no attribute "empno"

28/06/2019

Q)Write a Python Program to represent two accounts. Store Account data in computer memory and display Account details. Ensure that both the accounts have different state.

```
#account.py

class Account:

 def storeAccountData(self,accno,name,balance):

  self.accno=accno

  self.name=name

  self.balance=balance

 def displayAccountDetails(self):

  print("ACCNO:",self.accno)

  print("NAME:",self.name)

  print("BALANCE Rs",self.balance)

acc1=Account()

acc1.storeAccountData(100001,"Rama",40000)

print("First account details.......")

acc1.displayAccountDetails()

acc2=Account()

acc2.storeAccountData(100002,"Ramu",50000)

print("Second account details.......")

acc2.displayAccountDetails()
```

Q) What happens if displayAccountDetails() method is called without calling storeAccountData() method in "account.py"?

Ans:- AttributeError: 'Account' object has no attribute 'accno'

- **In the above Python application, Account object is created. But**
- **instance variables(attributes) are not created at the time of object creation.They are created in storeAccountData() method.**

Without calling that method, displayAccountDetails() method is called.

- In this method, "self.accno" causing the above error.
- A method is a self contained named block of code that can perform a specific task. There shouldn't be any restriction on the order of method calls. In any order they must be allowed to call.

Note:- It is not a good programming habit to define instance variables of a Python class in an instance method. We should have defined them in a constructor.

Q)What is a constructor?

- A specialized method of a class whose name is __init__() and that takes "self" reference as the first parameter is the constructor.
- PVM calls the constructor as soon as the object is created.
- After calling the constructor on the object, then only PVM gives the reference of the object to the application.
- Main purpose of a constructor is object initialiation. I.e. giving initial state to the object.
- In a Python class, instance variables creation and initializing those variables should be done in a constructor.
- constructor is the first method called on the object.
- In the life time of an object, constructor is called only once.

Q)What is the purpose of a constructor?

- Instance variables creation and their initialization is the main purpose of a constructor. I.e. Object initialization is the main purpose of a constructor.
- Whatever task we want to perform at the time of object creation, that task performing code can be placed in a constructor.

Q)Example program on constructor.

#accountconstructor.py

```
class Account:

 def __init__(self,accno,name,balance): #parameterised constructor
```

```
  self.accno=accno
  self.name=name
  self.balance=balance
 def displayAccountDetails(self):
  print("ACCNO:",self.accno)
  print("NAME:",self.name)
  print("BALANCE Rs",self.balance)
acc1=Account(100001,"Rama",40000)
print("Account details.......")
acc1.displayAccountDetails()
```

Q)Can object of Python class be created without writing a constructor in that class?

- **Constructor doesn't construct the object in Python.**
- **If no constructor is defined in a Python class, Cpython compiler writes a ZERO argument constructor during compilation phase, known as default constructor.**
- **Without calling the constructor, PVM doesn't return the reference of the object to the application.**

Q)How are constructors classified?

1.)Default Constructor(zero argument constructor)

2.)Parameterised constructor

Note:- Zero argument constructor is also called as default constructor

Because , if no constructor is explicitly written in a class, Compiler by default writes zero argument constructor only in the class.

For eg.

```
class Employee:
 def __init__(self):  # explicit default constructor
    print("Object created")
```

```
a=Employee() # PVM calls the explicit default constructor
```

Eg 2:-

```
class Account:
    pass
acc=Account() #PVM calls implicit default constuctor
```

Q)Can we overload a constructor?

- Having more than one constructor in a class with different number of parameters is known as constructor overloading.
- If we try to implement constructor overloading in a Python class, no complaint from CPython Compiler.
- However, only the last defined constructor is considered.
- Object creation time if number of arguments supplied is mismatched with the last defined constructor, error is reported.

Note:- Constructor overloading concept is not supported in Python.

```
class Account:
 def __init__(self,accno,name,balance): #parameterised constructor
  self.accno=accno
  self.name=name
  self.balance=balance

 def __init__(self): #explicit default constructor
  self.accno=1001
  self.name="Rama"
  self.balance=40000

 def displayAccountDetails(self):
  print("ACCNO:",self.accno)
```

```
  print("NAME:",self.name)
  print("BALANCE Rs",self.balance)


#acc1=Account(100001,"Rama",40000)
acc1=Account() #Zero argument constructor is called
print("Account details.......")
acc1.displayAccountDetails()
```

Q)What is the Syntax to create an object in Python?

```
        <class name>()
```

For eg. Employee()

Employee(1001,"Rama",40000)

Q)What is a destructor?

- A specialized instance method of a class whose name is __del__ andtakes "self" as the argument is the destructor.
- destructor is the last method that is called on the object justbefore the object is garbage collected.
- Resources allocated(if any) to the object in the constructor can be released in the destructor.

Q)What is garbage collection?

- Implicit deallocation of memory for the unreachable objects in a Python application is nothing but garbage collection.
- Garbage collector(background thread and is part of PVM) garbage collects the objects once they become unreachable.
- An application object becomes unreachable when no references are referencing that object.

30/06/19

Q.)Explain about range data type in Python:

- Range represent immutable index sequence of integer

- range function it is used to create range type object.

  R=range(5) #range type object is created with name r

- it is a single argument supplied for range function it construct a sequence of integers from 0 to n where n is argument

  for i in range(5):

    print(i)   # 0 1 2 3 4

- two arguments are supplied for range function is construct a sequence of integer between a specified arguments

  r=range(1,6)

  for I in r:

   print(I)   # 1 2 3 4 5

- Three arguments are supplied for range function construct a sequence with the specified range with step specified in third argument.

  for i in range (1,10,2)

   print(i)   # 1 3 5 7 9

- in general range is used for loop to perform a task a specified number of time (fixed number of iteration)

Note:- we can specify negative number of third argument in range function.

Eg. range(5,1,-1)   # build a sequence 5 4 3 2

- We can access the element of range using index also.

  B=range(5)

  B[0]=5   #error, because range is immutable

Q.)What are the built-in collection types data types in Python?

- List
- Tuple
- Set
- Dict

Q.)What is a list in Python?

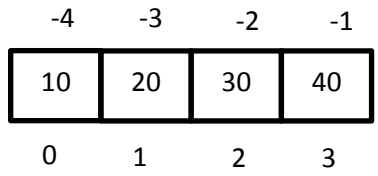
- **List is built in data structure in Python.**
- **A list is in an index order mutable collection.**
- **List object is resizable (both growable and shrinkable).**
- **Heterogeneous element can be stored in list type of collection.**
- **Insertion order is preserved.**
- **Duplicates are allowed.**

Q.)How to create a list?

- **Statically by using square brackets[]**

  l=[] #empty list

  l=[10,20,30,40]



- **Dynamically through user input**

l=eval(input("Enter the list of items")) #user enters [10,20,30]

- **by using list function**

  l=list(range(5)) #[0,1,2,3,4]

  l=list("hello") #['h','e','l','l','o']

- **By using list comprehension**
- **By using cloning**

Q.)Explain about list comprehension?

- **a comprehension is a construct that facilitates the creation of one sequence using another sequence**
- **List comprehension has the following syntax**

[expression for element in sequence]

For eg

l=[x for x in range (1,6)] #[1,2,3,4,5] is built

```
l=[x for x in range (1,6)if x%2==0]     #[2,4] is built
```

- List comprehension is concise way of creating a list.

Q.)Explain about cloning of list?

- Creating the exact copy of list is known as cloning
- A list can be cloned two ways

1. Using size operator
2. Using copy method of list object

```
l=[10,20,30,40]

nlist=l[:]    #using slice operator cloning is done.

nlist2=l.copy()    #using copy method cloning is done.
```

Q.)How to access the list of element?

1. Using index
2. using a loop (for loop and while loop)
3. Slicing

```
Accessing the element of list using index

l=[10,20,30,40]

l[0] #10

l[-1]    #40

l[-3]    #20

l[5]     #error, index out of range


Accessing the element of list using while

l=[10,20,30,40]

leng=len(l)

i=0

while (i<leng):

  print(l[i])

  i=i+1
```

Accessing the element of list using for loop

```
l=[10,20,30,40]
for i in range l:
  print(i)
```

Note:- accessing each element of the list sequentially is known as traversing of list.

Accessing element of list using slicing

- **Extracting a portion of the list is known as slicing**
- **Slice operator(:) used to perform slicing.**
- **Slice operator will be the following form.**

  [Start:Stop:Step]

- **Start specifies from where slicing should be start default is 0**
- **Stop specify slicing should be up to what index excluding the last.**
- **Step specify the increment of the index default is 1.**

```
l=[10,20,30,40,50,60,70]
l[:2]    # [10,20]
l[::-1]  # [70,60,50,40,30,20,10]
l[-3:]   # [50,60,70]
l[2::2]  # [30,50,70]
```

Q.)What is the output of following code:

```
a=[10,20,30]
b=[40,50,60]
c=a+b
print(c)
```

Ans:- **[10,20,30,40,50,60]**

- **when the operands are list "+" operator acts as concatenation operator instead of arithmetic addition operator.**

Q.)What is output of following code:

```
a=[10,20,30]
y=a*3
print(y)
```

ans:- `a=[10,20,30,10,20,30,10,20,30]`

- **when one operand is list and another operand is an integer "*" act as repetition operator.**

Q.)How to find if an element is present in a list?

Ans:-Using membership operator (in).

Q.)What is a nested list?

- **If a list is the element of another list it called as nested list.**

```
l1=[10,20]
l2=[40,l1]    #l1 is in nested list
```

Q.)What is aliasing a list?

Ans:-Giving another name to a list is known as aliasing list

```
l1=[10,20]
l=l1     #list is another name now
print(l)
```

Q.)What is the output of following code:

```
l=[10,20]
l[0]=30
print(l)      #[30,20]
```

Q.)What are the important building function to deal with list?

1. **Sum():-add the element of list if they are numbers and return the sum.**
2. **Min():- return the smallest element of the collection**
3. **Max():- return the largest element of a list**
4. **Len():-return the number of element in the list**

Q.)What is the output of following code:

```
l=[10,20,30,40]
print(min(l))      #10
print(max(l))      #40
print(sum(l))      #100
print(len(l))      #4
```

Q.)What are command line arguments in Python

- **Value supplied to the Python application from the command line while executing that application are known as command line argument.**
- **Python interpreter store the supplied argument as a string in list whose name is "argv". "argv" belong to built in module "sys".**

Q.)Write a python script that display all the command line arguments:

```
#commandlineargument.py
#py commandlineargument.py 10 20 30
from sys import argv
list=argv[1:]      #to skip file name
for i in list:
  print(i)
```

or

```
from sys import argv
i=1
n=len(argv)
while i<n:
  print(argv[i])
  i=i+1
```

Q.)write a python script that add all the members applied at the command line and display the sum

```
#addcommandlineargument.py

#py addcommandlineargument.py 10 20 30

from sys import argv

list=argv[1:]      #to skip file name

sum=0

for i in list:

  sum=sum+int(i)

print("the sum is:",sum)
```

Q.)What is the purpose of command line argument?

- **certain value of example username and password should be hard coded in the application**
- **To supply different value to the application at different time of the execution of the application and there by preventing the hardcoding we use command line argument.**

Q.)What are the important libraries method of list?

- **Append():- this library method is used to add an element to the end of list.**

  ```
  l=[10,20]

  l.append(30)

  print(l)      #[10,20,30]
  ```

- **Insert():- used to add an element at a specific location.**

  ```
  l=[10,20]

  l.insert(1,15)

  print(l)      #[10,15,20]
  ```

- **Counter():- return the number of occurrence of an element in list**

  ```
  l=[10,20,30,10]
  ```

```
l.count(10)   #2
```

- Extend():- used to add on list to another list

```
l1=[10,20]

l2=[30,40]

l1.extend(l2)

print(l1)     #[10,20,30,40]
```

- Index():- it return the index of first occurrence of a specified element in list.

```
l=[10,20,30,10]

l.index(30)   #2

l.index(10)   #0
```

- Pop():- delete element from the list it return the element to the application this method takes the index of element to be delete an argument
- Remove():- error the specified element delete element it doesn't return the to the application if specified element is not there in the list it rise error
- Copy():- used to cloning
- Reverse():- reverse the list
- Sort():- used for sorting a list

```
l.sort()

l.sort(reverse=True)   #for resverse
```

- Clear():- this method is used to empty the list of IE to remove all the element of list

```
l=[10,20,30,10]

l.clear()

print(l)      #[]
```

Q.)What is tuple?

- Tuple is immutable ordered collection

- **Tuple is similar to a list except immutability I.E. tuple is read only list.**

Q.)How to create a tuple?

1. **Statically**

   t=(10,20)

   t=() #empty tuple

2. **dynamically by reading value**

   t=eval(input("Enter the values:")) #user enters (10,20,30)

3. **using tuple function**

   t=tuple([10,20])

   t=tuple(range(5))

Note:- tuple comprehension doesn't create a tuple. Cloning also is not support to create a tuple.

Q.)How to access the element of tuple

Ans:-Similar to that of list.

Q.)What is the output of following code:

```
t=(10,20)
t[0]=40
print(t)
```

Ans:- error, tuple is immutable cant modify its content.

Q.)What are the building function used with tuple.

sum(), min(), max(), len(), sorted()

Q.)Explain about tuple packing and unpacking

- **Storing normal variable value into tuple is known as packing.**

Eg

```
a=10
b=20
```

```
t=a,b

print(t) #(10,20)
```

- storing the content of tuple into non collection type variable is nothing but unpacking

eg

```
t=(10,20,30)

a,b,c=t
```

Note:- number of variable should be match that of number of value in tuple during unpacking.

01/07/2019

Q)What are the similarities between list and tuple?

- Both are built-in data structures of Python
- Both are ordered collections(indexed based)
- Both can hold hetrogeneous elements
- Duplicate elements can be stored in both of them.
- insertion order is preserved in both of them.
- Accessing of elements is similar in both of them.

Q)What are the differences between list and tuple?

List	*Tuple*
Mutable	*immutable*
items are enclosed between square brackets []	*items are enclosed between square brackets ()*
Can't be used as key for a dictionary	*Can be used*
when contents are required to be updated frequently, we go for List.	*updation not required, read-only; we go for Tuple*
can be cloned	*can't be*
can be createdusing comprehension.	*Cant’t be*

Q)What is a Set?

- An unordered, mutable collection is a Set.
- set is one of the built-in data structures provided in Python.
- Duplicates are not allowed.

- Hetrogeneous elements are allowed.
- Insertion order is not preserved.

Q)How to create a Set?

1. Using flower brases, a predefined collection(statically)

```
s={10,20,30}
```

2. Using set() function.

```
s=set() #empty set is created
s=set([10,20,30,40])  # {10,20,30}
```

3. Dynamically by reading values from the keyboard

```
s=eval(input("Enter the elements of a set:")) #
user enters {30,30,40,60}
```

4. using comprehension

```
s={x for x in range(5,10)}
```

5. cloning

```
s={10,20,30,40}
s1=s.copy() # exact copy of the above set is created
```

Q)How to access the elements of a Set?

> Through traversal.

Note:- Indexing and slicing is not possible with Set to access its elements.

```
s={10,20,30,40}
for x in s:
  print(x)   # tranversing the Set
```

Q)How to know if an element belongs to a Set?

> Using membership operator.

```
40 in s # returns True
```

Q)What are the library functions that can be used with a Set?

1. min():- returns the smallest element of the set

2. max():- returns the largest element
3. len():- returns the number of elements in the Set
4. sum():- Returns the sum of the elements of the Set

Q)What are the library methods of Set object?

- add():-adds an element to the Set.

```
s={10,20}
s.add(30)
s #{10,20,30}
```

- update():- used to add a collection of elements to the Set.

```
s.update(range(1,5)) # 1 2 3 4 added to Set
```

- copy() :- for cloning
- pop():- to delete an orbitary element and returns the same
- remove():- to delete the specified element. If not there, error
- discard():- deletes the specified element. No error even if the element doesn't exist
- clear():- deletes all the elements
- union():- returns a Set with all the elements of both the sets

```
s1={10,20,30}
s2={10,45,67,56}
s1.union(s2) #  {10,20,30,45,56,67}
```

- intersection() :- returns a Set with common elements of both the sets

```
s1.intersection(s2) #{10}
```

- difference() :- returns a set that contains elements present in the first set

```
but not in the second set
s1.difference(s2) #{20,30}
```

- symmetric_difference():-Returns a Set with non common elements of both the Sets.

Q)What is a frozenset?

- Read-only version of Set. I.e. an immutble Set.

```
s={1,2,3,4} #mutable
s1=frozenset(s) # s1 is immutable {1,2,3,4}
```

Q)What is a Dictionary(dict type) in Python?

- A mutable, unordered collection of key,value pairs is nothing but a Dictionary(dict type).
- Dictionary is one of the built-in data structures provided by Python.
- Hetrogenerous elements are allowed
- Insertion ordered is not preserved.
- Keys can't be duplicated. But values can be duplicated.
- It is a dynamic data structure(resizable. I.e. both growable and shrinkable).

Q)How to create a Dictionary in a Python Program?

1) d={} #empty dict object created

2)d={1001:"Rama",1002:"David",1003:"Rahim"}

3)Dynamically by taking data from the keyboard.

```
d=eval(input("Enter key,value pairs:"))
```

4)Using cloning.

```
d2=d.copy() # new dictionary is created.
```

Q)How to access data from the dictionary?

Note:- Slicing and indexing is not allowed on Dictionary.

1.)Individual data item can be retrieved by calling

get() method.

For eg.

```
d.get(1001) #"Rama"
```

```
d.get(1003) #Rahim
```

2.)Through traversal .

```
for k,v in d.items():
print(k,":",v)
```

Q)How to delete the elements of a dictionary

1. del d[1001] # 1001:Rama entry is deleted.
2. d.pop(1003) # 1003:Rahim entry is deleted
3. d.popitem() # an orbitary element is deleted.

Q)What are the library functions that can be used with Dictionary?

```
min(),max(),len()
```

Q)What are the library methods of dict object?

- clear():- deletes all the elements of the dictionary
- get():- To retrieve an element from the dictionary
- pop():- to delete a specified elemenet.

```
d.pop(1001)
```

- popitem():- deletes an orbitary element.
- keys() :- Returns all the keys

```
for k in d.keys()
print(k) # 1001 1002 1003
```

- values():- returns all the values

```
for v in d.values()
print(v) #Rama David Rahim
```

- items():- Returns a list of tuples with key,value pairs.

```
[(1001,"Rama"),(1002,"David"),(1003,"Rahim")]
```

- copy():- for cloning
- setdefault():- makes a new entry if the specified key, value pair is not there in the dictionary. If there, it returns the value (acts like get())

For eg.

```
d.setdefault(1004,"XYZ") # stores an element

d.setdfault(1001,"ABC") #returns "Rama"
```

- update():- Used to add the specified dictionary elements to the current dictionary.

```
d2={1005:"LMN",1006:"KJH"}

d.update(d2) # d2 elements added to d
```

Q)Explain about "bytes" type

- "bytes" type is used to represent an immutable indexed collection of integers that range between 0-255
- By using bytes() library function, "bytes" object is created.

  For eg.

```
b=bytes([2,3,4])
```

- In networking applications where large files are uploaded and to represent the pixels of images, "bytes" type is used.

Q)Explain about "bytearray" type.

- "bytes" type is used to represent a mutable indexed collection of integers that range between 0-255
- By using bytearray() library function, "bytearray" object is created.

02/07/2019

Q)What is the output of the following Python program?

```
#empcount.py

class Employee:

  def __init__(self):

   self.emp_count=0

   print("Employee appointed....")

   self.emp_count +=1
```

```
  def getEmployeeCount(self):
   print("Total number of employees appointed:",self.emp_count)


e1=Employee()
e1.getEmployeeCount()


e2=Employee()
e2.getEmployeeCount()
```

Ans:-

```
Employee appointed....
Total number of employees appointed:1
Employee appointed....
Total number of employees appointed:1
```

- Reason for the unexpected output is,we are trying use an instance variable(emp_count) to represent whole class level information (total number of objects created).
- Instance variables should be used to represent individual object level information.
- To represent class level information we have to use "class(level)variables" in Python.

Q) Modify the previous "empcount.py" so as to represent class level information.

```
#empcount2.py
class Employee:
  emp_count=0 #class level variable
  def __init__(self):
   print("Employee appointed....")
   Employee.emp_count +=1
```

```
  def getEmployeeCount(self):
   print("Total number of employees appointed:",Employee.emp_count)


e1=Employee()
e1.getEmployeeCount()


e2=Employee()
e2.getEmployeeCount()
```

Q)What are the different kinds of variables we encounter in OOP through Python?

1. **instance(level) variables**
2. **class (level) variables**
3. **local variables**

Variables defined within the methods of the class are local variables. They are no way associated with the object.

Q)Explain about class level variables.

- **variables defined in the class and outside the methods of the class are by default class level variables.**
- **As soon as the class is loaded into memory and before any object is created, memory is allocated for class level variables.**
- **For entire class only one copy of class level variable is created irrespective of the number of objects created.**
- **Every object can share the class level variable but can't own it.**
- **class level variables are used to represent entire class level information.**
- **A class level variable can be accessed by using class name.**

For eg.

Employee.emp_count

- **class level variable can be accessed through object reference also. But, if we try to modify the value of class level**

variable, it doesn't affect the class level variable. Instead, a new instance variable is dynamically attached to the object.

For eg. in the previous program, if we say,

e1.emp_count=10 # creates instance variable for e1

Q)Explain about class(level) methods in Python.

- A method of a class that is applied with a built-in decorator, @classmethod and that takes class reference as the first argument is nothing but a class method.
- A class method is called using the class name.
- Generally, class methods are used to deal with class level variables.

Q)How to hide data of a class outside the class?

- By using private variable concept.
- "private" keyword is not there in Python.
- If a variable or a method name is preceded with double underscores, it acts as private member of the class. It is not visible(accessible) outside the class.

```
#empcount3.py

class Employee:

  __emp_count=0 #class level  private variable

  def __init__(self):

   print("Employee appointed....")

   Employee.__emp_count +=1 #accessing class level variable using class name

  @classmethod

  def getEmployeeCount(cls): #class method

   print("Total number of employees appointed:",cls.__emp_count)

Employee.getEmployeeCount()#calling the class method using class name

e1=Employee()
```

e1.getEmployeeCount()

e2=Employee()

e2.getEmployeeCount()

Q)Explain about static methods in Python?

- **A method of a class that is decorated with @staticmethod built-in decorator is known as a static method.**
- **A static method doesn't get class reference unlike class method and also doesn't get object reference unlike instance method.**
- **static methods do not know about object and class. They are mere utility methods.**
- **A static method can be called using class name.**

```
class Data:
 @staticmethod
 def  search():   #defining the static method in Data class
              .......
  Data.search() # static method calling
```

03/07/2019

Q)What is inheritance?

- **Inheritance is a mechanism of acquiring features from the existing entities.**
- **The mechanism of creating new classes from already existing classes using is-a relationship is nothing but inheritance.**
- **Already existing classes are known as parent classes or base classes.**
- **Newly created classes are known as child classes or derived classes.**
- **Derived classes inherit properties(varibles)and behaviour (methods) from the base classes.**

- **Base classes are generalised classes as they have general properties and behaviour common to all its derived classes.**
- **Derived classes are specialized classes.**
- **Inheritance provides code reusability.**

Q)What is the syntax to create a derived class(child class) in Python?

```
class    <derived class name>(<base class name>):
```

for eg.

```
class Employee(Person):
    .......
  .......
```

Q)Write a Python program in which inheritance is implemented.

#studentperson.py

```
class Person: #base class
 def displayPersonDetails(self):
  print("Displays person details.......")
class Student(Person): #derived class
 def displayStudentDetails(self):
  print("displays student details...")
s=Student()
s.displayPersonDetails()#inherited method is called on child object
```

#studentperson2.py

```
class Person: #base class
 def __init__(self,name,age):
  self.name=name
  self.age=age
 def displayPersonDetails(self):
  print("NAME:",self.name)
```

```
  print("AGE:",self.age)
class Student(Person): #derived class
 def __init__(self,name,age,rollno,marks):
  super().__init__(name,age) #child constructor calling parent constructor
  self.rollno=rollno #instance variable creation
  self.marks=marks

 def displayStudentDetails(self):
  self.displayPersonDetails()#inherited method
  print("ROLLNO:",self.rollno)
  print("MARKS:",self.marks)

s=Student("Rama",21,100001,78)
s.displayStudentDetails()
```

Q)Explain about Access modifiers in Python.

- An access modifier specifies the accessibility mode(visibility mode) of the members of a class.
- If a member of a class is "private", it is visible/accessibile only within the methods of that class.
- If a member of a class is "protected", it is accessibile ony within the methods of same class and its derived classes.
- If a member of a class is "public", it is accessible in the entire application.
- In Python, by default, all the members of a class are "public".
- Members of a class if preceded with single underscore are considered to be protected.
- Members of a class preceded with double underscores are considered to be private.
- Python doesn't support access modifiers concept.
- Only convention. But no restriction.

Q)What is method overriding?

- **Redefining the parent class method in child class is nothing but method overriding.**
- **When method overriding is implemented, parent class method is known as "overridden" method and child class method is called "overriding" method.**
- **When method overriding is implemented and the method is called on the child class object, "overriding" method only is executed. i.e. child class method only is executed.**
- **method overriding is implemented in 3 scenarios**

1. **parent provided method name reusability required but not code reusability**
2. **parent provided functionality insufficient.**
3. **parent has provided only method name but no code(abstract method).**

```
        #methodoverriding1.py
class Vehicle:#parent class
 def move(self): #overridden method
  print("every vehicle should move this way")
class Car(Vehicle): #child class
 def move(self): #overriding method
  print("As a car I move in my own way")
c=Car() #child class object creation
c.move() #child version of move only is executed


#methodoverriding2.py
class Vehicle:#parent class
 def move(self): #overridden method
  print("every vehicle should move this way")
class Car(Vehicle): #child class
 def move(self): #overriding method
```

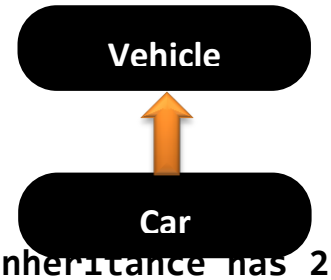
```
  super().move() #calling overridden method
  print("As a car I move in my own way also")
c=Car() #child class object creation
c.move()
```

Q)What are the types of inheritance?

- Inheritance is of two types.

1. single inheritance
2. multiple inheritance

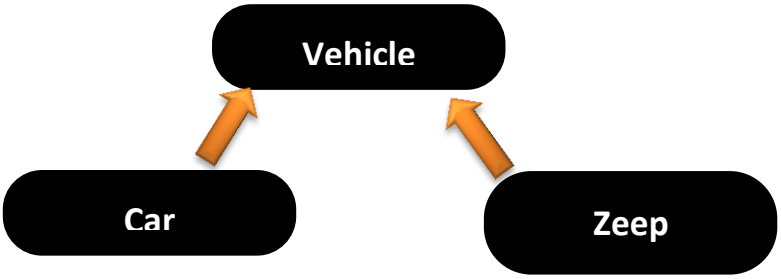
- Single inheritance is that form of inheritance in which a child class inherits from only one parent class.
-



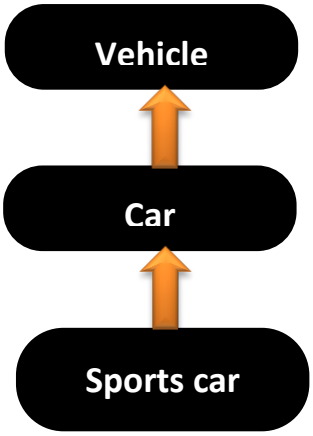
- Single inheritance has 2 special cases.

1. hierarchical inheritance
2. multi-level inheritance

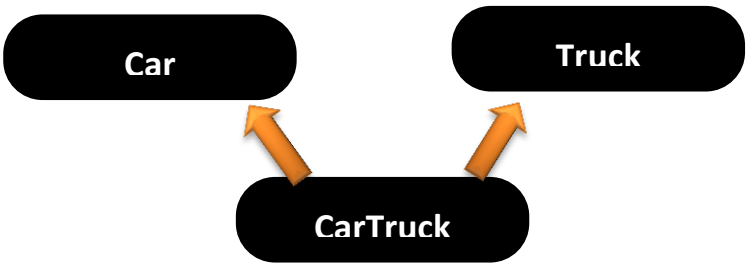
- Hierarchical inheritance is that form of inheritance in which a parent
- class has multiple child classes.



- Multi-level inheritance is that form of inheritance in which a derived class acts as base class for another derived class



- Multiple inheritance is that form of inheritance in which a child class inherits from more than one parent class.
- whenever one hierarchy of classes is not providing required functionality, a child class inherits from another level of hierarchy which becomes multiple inheritance.



Note:- Python supports both single inheritance and multiple inheritance

Q)Example program on multiple inheritance.

```
 #multipleinheritance.py

class Car:

 def moveFast(self):

  print("moves faster")

class Truck:

 def bearLoad(self):

  print("bears the load")

class CarTruck(Car,Truck): # child class inheriting from 2 parent classes

 pass

ct=CarTruck()#child class object creation

ct.moveFast()

ct.bearLoad()
```



Note:- While implementing multiple inheritance, if the parent classes

have method with same name, method of that parent class is executed which is specified first.

```
#multipleinheritance2.py
class Car:
 def move(self):
  print("moving like car")
class Truck:
 def move(self):
  print("moving like Truck")
class CarTruck(Car,Truck):
 pass
ct=CarTruck()#child class object creation
ct.move() #Car version of move() executed
```

04/07/2019

Q)What is an abstract method?

- **A bodyless method(null body method) that has only method header and doesn't have implementation is nothing but abstract method.**
- **An abstract method is decorated with built-in decorator, @abstractmethod**
- **In parent classes abstract methods are declared to enforce method implementation by their child classes.**

```
#abstractmethodexample.py
from abc import *  #all the members of "abc" module available
class Vehicle:
 @abstractmethod
 def move():pass
```

Note:- a method with body is known as concrete method(complete in implementation). A method without body is abstract(incomplete) method.

Q)What is an abstract class?

- **A Python class becomes abstract if**

1. **it inherits ABC class of "abc" module**
2. **it has atleast one abstract method**

- **An abstract class can't be instantiated. Only concrete class can be instantiated.**

```
#abstractclassexample.py

from abc import *

class Vehicle(ABC):

 @abstractmethod

 def move():pass

class Car(Vehicle):pass

c=Car()#error. must provide body for move()
```

Note:- Whenever a class is inheriting an abstract class, it is the responsibility of that child class to override all the abstract methods of the parent class. otherwise it also becomes abstract and can't be instantiated.

- **An abstract class can have all kinds of variables and methods for children sake in addition to abstract methods.**

Q)Explain about super() .

- **super() is a built-in function used to access base class methods &constructors in derived class.**
- **From child class methods, parent class variables can't be accessed using super() should use "self" for this.**
- **From child class methods, parent class level variables can be accessed using super()**
- **from child class methods, parent class instance method, class method and static method can be called using super()**

Q) What are the different types of relationships among the objects of an object oriented system?

1.)has-a relationship

2.)is-a relationship

3.)use-a relationship

- **If the reference of an object is made as the instance variable of another class(object),**

  it becomes has-a relationship.

- **If the reference of an object is supplied as an argument to the method of another Object(class), it is "use-a" relationship.**
- **Parent-child relationship forms "is-a" relationship.**

Q) How to display the state of an object by supplying the object reference as argument to print() function?

- **By overridingng __str__() method in the class, we can achieve this.**
- **If we don't override this method, when object reference is supplied as argument to print() function, parent class(__builtins__.object class)**
- **__str__() method is executed and therefore object address is printed rather than object data.**
- **This method should return a string(textual representation of object state)**

Q)What is the base class of all classes in Python?

"object" class of __builtins__ module

05/07/2019

Q)What is Polymorphism?

- **An entity behaving differently in different context is nothing but that entity is exhibiting polymorphism.**
- **"one interace multiple forms" is the key feature of polymorphism**

- **Polymorphism provides extensibility(of code).**

Q)How is polymorphism implemented in an object oreinted system?

- **Using overloading concept.**

Q)What is method overloading?

- **Within the same class having multiple methods with the same name but with different signature is nothing but method overloading.**

Q)Example Python program on method overloading.

```
class Addition:
 def add(self,n1,n2,n3):
  return n1+n2+n3
 def add(self, n1,n2):
  return n1+n2


a=Addition()
sum=a.add(10,20) #works
print("The sum is:",sum)
sum=a.add(10,20,30) #error
```

Note:- In 'Addition" class we tried to implement method overloading. No error comes. But only last method is taken into consideration by Python(platform). method calling time, considered method arguments are matching, it gets executed. If not matching, error is reported.

Q)Does Python support method overloading?

- **No**

Q)What is operator overloading?

- **Using an operator for more than one functionality is nothing but operator overloading.**

- Python implements polymorphism through operator overloading.

Q)Example program on operator overloading.

```
#bookplusoverloading.py
class Book:
 def __init__(self,title,price):
  self.title=title
  self.price=price
 def __add__(self,b):
  return f'The price of both the books:{self.price+b.price}'
b1=Book("Python Complete Reference",635)
b2=Book("Mastering Python",1275)
print(b1+b2)
```

Note:- In the above example, "+" operator is overloaded by overriding __add__ method of "object" class. For each operator of Python ,corresponding method is given in "object" class known as magic methods.

For eg.

>	__gt__
-	__sub__
<=	__le__

Exception handling

Q.) what are the different type of errors encounter during development and execution of python program?

1. Syntactical error
2. Runtime error
3. Logical error

Q.) what is an exception?

- Runtime error is nothing but an exception.

- An abnormal event that is occure during the execution of a python program which disrupt the normal flow of the application is nothing but an exception.
- Exception is an object in python.

06/07/2019

Q.) what are the danger caused by an exception?

- When an exception is raised, application terminated abnormally.
- Abnormally terminated an application caused the following dangers:
  - User's data/work is lost
  - User has no information why application was crashed suddenly
  - Resources allocated to the application can't be released gracefully.

Q.) what is an exception handling?

- Exception handling is an mechanism to deal with exception and there by preventing the dangers caused by exceptions.
- Exception handling mechanism makes python application robust.

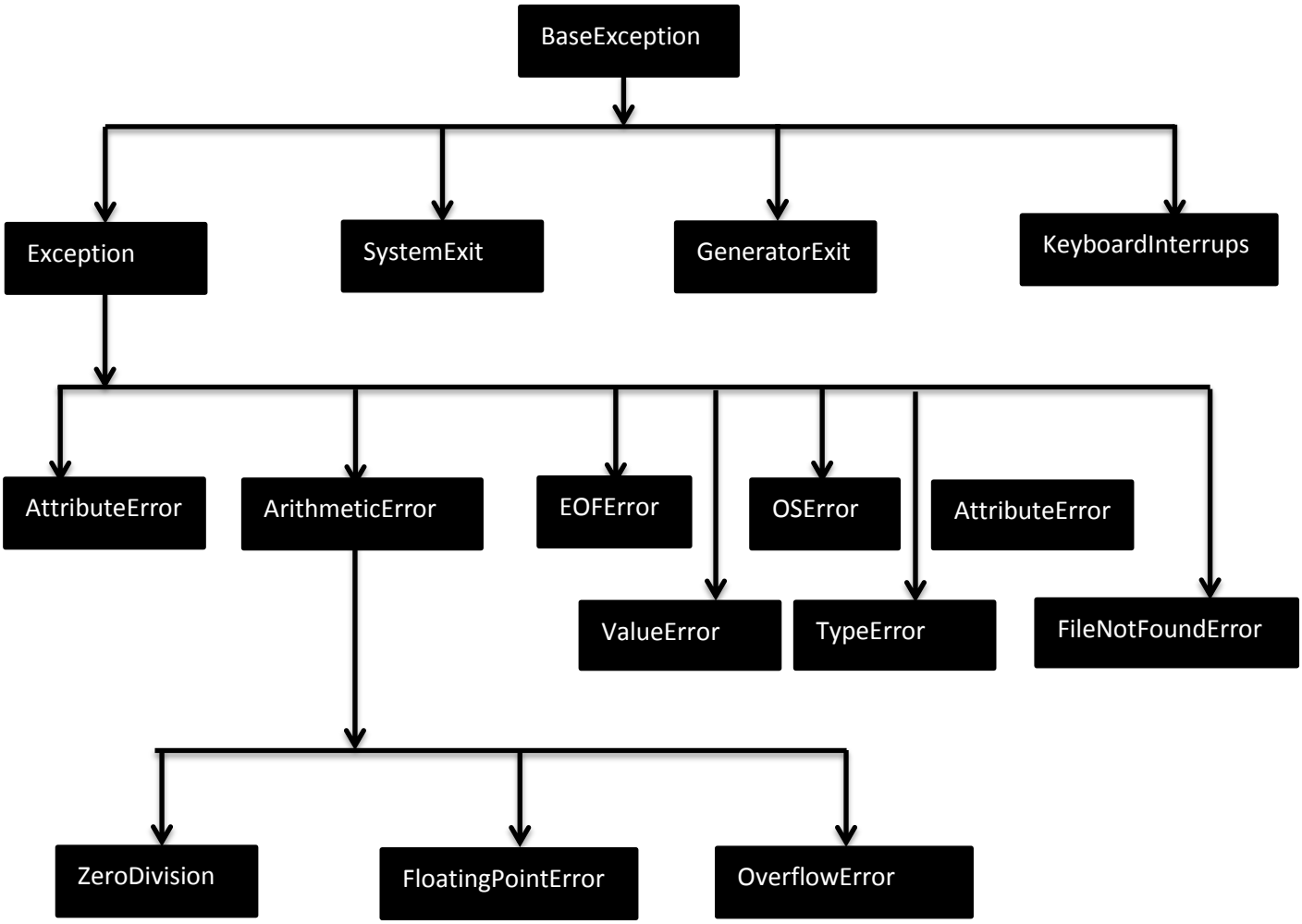
Note:- robust is ability stand with failure.

Q.) how does python support implement exception handling in python application?

- Python provides language level support to fecilitates the programmer to implement exception handling.
- As far as language support is concern, python language has provided the following keyword to implement exception handling
  - Try
  - Except
  - Raise
  - Finally
  - Else
- As library support, python library has provided built-in classes to represent every abnormal scenario during the execution python program.

Q.) give the hierarchy of python standard(built-in) exception(classes).

- **Parent class (base class) of all built-in exception calss in python is base exception**
- **It has 4 child classes.**
- **Python application developers almost all the time deals with only the exception of type "exception" and it child classes.**



Q.) write a python program that terminated abnormally if exception is raised by not handled.

```
#withouthandling.py

print("Python application is started")

n=int(input("enter first number"))

d=int(input(enter second number))

q=1

q=n/d #if d is zero will through exception

print("Result is",q)
```

print(“Python program executed successfully”)

Q.) Explain about “try” keyword.

- **Try is a exception handling keyword given in python.**
- **Try is used to create a block of code.**
- **Try is used to place doubt full code that may cause exception.**
- **Writing try block does not mean that we are trying for exception, instead we are trying to handle the exception if they get raised.**
- **Keep application code in try block is for monitoring the risky code to deal with exceptions that may be caused by that code.**
- **If exception is raised in try block instead of programing getting terminated remaining code after try block, after re pointing of the exception raising get terminated.**

Q.) explain about “Except” keyword.

- **Except is on of exception handling keyword given in python language.**
- **Except is used to create a block of code.**
- **Exception handling code is placed in except block an hence is called as exception handler.**
- **If exception is raised in try block control comes to except block.**
- **If exception is not raised in try block control is not comes to except block.**
- **For one try block we can write any number of except block each except block handle one type of exception in general.**

Note:- between try and except no other code is allowed.

07/08/2019

Q.) Modify the previous application(withouthandling.py)so as to implement exception handling.

```
#withhandling.py

print("Python application commenced")

n=int(input("Enter the first number:"))
```

```
d=int(input("Enter the second  number:"))
quotient=1 #identity of division
try:
 quotient=n/d
 print("Result of division is:",quotient)
except ZeroDivisionError:
 print("Division failed.Second number should not be zero")
print("Python application successfully executed")
```

Note:- In except block we can get the reference of the exception object.

```
except ZeroDivisionError as msg:
  print("Division failed. Second number can't be zero.")
  print(msg)
```

- **We can display location details of the raised exception also as follows.**

```
except ZeroDivisionError :
 print("Division failed. Second number can't be zero.")
 traceback.print_exc()
```

Note:- We need to import "traceback" module in the program.

```
import traceback
```

Q.) What is default exception handler in Python?

- **"except" block without mentioning the name of the exception class is the default exception handler. It can handle any type of exception and we can give any generic message in this block.**

```
except : #default exception handler
```

```
 print("Division failed. Second number can't be zero.")
```

Q.) How many "except" blocks can be associated with one try block?

- **There is no limitation. We can have any number of "except" blocks associated with one "try" block.**
- **Risky code may cause what kind of exception, to address/handle each type of exception individually, we write one except block.**

Note:- When multiple except blocks are written for one try block, default exception handler should be placed last.Otherwise, it is a syntactical error.

Q)Write a Python program in which multiple "except" blocks are associated with one try block.

```
#multiexcept.py

print("Python application commenced")

try:

 n=int(input("Enter the first number:"))

 d=int(input("Enter the second  number:"))

 quotient=1 #identity of division

 quotient=n/d

 print("Result of division is:",quotient)

except ValueError:

 print("Division failed.Please enter Digits only")

except ZeroDivisionError:

 print("Division failed.Second number should not be zero")

except:

 print("Division failed.Invalid input")

print("Python application successfully executed")
```

Note:- Within one "except" block also multiple exceptions can be handled.

```
except (ZeroDivisionError,ValueError)as msg:
```

```
  if isinstance(msg,ZeroDivisionError):
   print("Division failed. Second number can't be zero")
  else:
   print("Division failed. Please enter only numbers")
```

Q.) Explain about "finally" keyword.

- **"finally" keyword is used to create a block of statements.**
- **Nature of finally block is that it gets executed in exception raised case and also in not raised case.**
- **Resources releasing code is placed in finally block.**

For eg. file closing

Q.) Can we have a try block without except block?

- **Yes.**
- **finally block can be placed after try block if except block is not there.**

Note:- finally block should be preceded either with except block or with try block.

Q.) Explain about "raise" keword.

- **Normally, exceptions are raised by PVM.**
- **Sometimes, application also may need to raise exceptions explicitly.**
- **"raise" keyword purpose is to raise exceptions explicitly in the Python Application.**

For eg. raise InsufficientFundsException

Q.) Explain about "else" keyword in the context of exception handling.

- **"else" block can be associated with exception handling.**
- **It is an optional block.**
- **It should be placed after except block and before finally block.**
- **else block is executed only when try block is completely executed without raising any exception.**

```
try:
   n=int(input("Enter denominator for division:"))
   print(10/n)
except:
   print("Don't make denominator zero")
else:
   print("division successful")
finally:
   print("program executed successfully")
```

Q.) Explain about nested try block

- **Placing a try block inside another try block is nothing but nesting of try blocks.**

```
try:  #outer try
 try:  #inner
  ....
 except: #inner
  .....
except:  #except
 .....
```

Note:- If matching inner except block is not there for inner try, program is not terminated. outer except block is considered.

Q)Explain about user defined exception.

- **Our own created exception i.e. programmer created exception class is nothing but a user defined exception.**
- **User defined exception class must inherit built-in "Exeption" class or its child class.**
- **Whenever application rule(business rule) is violated, if we want to stop further processing instantaneously, user defined exception is the best solution.**

Q) Write a Python program in which, user defined exception is implemented.

```
#userexception.py

class VotingException(Exception):

 pass

def isEligibleToVote(age):

 if age<18:

  raise VotingException

 print("eligible to vote")

print("voting application starts.....")

age=int(input("Enter the age:"))

try:

 isEligibleToVote(age)

except VotingException:

 print("Invalid age.Can't vote")

  print("voting application completely executed.....")
```

File Handling

Q.) What is a file?

- **Named memory location of secondary memory(hard disk) is nothing but a file OR disk file.**

Q.) What is file handling?

- **Performing I/O operations with the file is nothing but file handling.**

Q.) How to implement file handling in a Python application?

- **File is a named memory location in the secondary memory(hard disk).**

- **Performing Input/Output Operations with the file is nothing but File Handling in general.**
- **Three steps are involved in file handling in a Python Application.**

Step 1:- open a file

Step 2:- perform read/write operation

Step 3:- close the file once done with the I/O Operation

Q.) Write a Python program to store some text into a text file.

```
#writefile.py

file=open("one.txt","w")

file.write("File handling today")

file.close()
```

Q.) Write a Python Program to read text from text file and display on the console.

```
#readfile.py

filename=input("Which file to read and display:")

f=open(filename,"r")

data=f.read()

print(data)

f.close()
```

Q.) Write a Python Program to copy the contents of one text file into another text file.

```
#filecopy.py

filename1=input("Enter the source file name:")

filename2=input("Enter the target file name:")

file1=open(filename1,"r")

file2=open(filename2,"w")

data=file1.read()
```

```
file2.write(data)

file1.close()

file2.close()

print(file1.name,"is copied successfully into",file2.name)
```

Q.) Write a Python Program to copy the contents of one image file into another image file.

```
#imagecopy.py

file1=open("tiger.jpeg","rb")

file2=open("tiger2.jpeg","wb")

bdata=file1.read()

print(type(bdata))

file2.write(bdata)

print("Image file copied successfully")

file1.close()

file2.close()
```

Q.) What are the different file modes?

1.) r- read mode. If the specified file is not available in this mode,

FileNotFoundError is raised.

2.)w-write mode. If the specified file is not available,new file is created. If available, its contents are overwritten.

3.)a-append mode. similar to write mode except without erasing the previous content, new content is added at the end.

4.)r+- read mode with write ability

5.)w+- also can read

6.)a+ readable as well

7.)x exclusive mode for file creation. If file alreay exists FileExistsError is raised

Note:- All above modes are for text files. For binary files we have to specify "b" at the end.

For eg. rb,wb,w+b etc.

Q.) Modify readfile.py(file reading program) so as to implement exception handling completely

```
#readfileexcp.py

filename=input("Which file to read and display:")

f=None

try:

 f=open(filename,"r")

 data=f.read()

 raise IOError # intentional raising of error after file is opened

 print(data)

except FileNotFoundError:

 print("Specified file doesn't exist")

except:

 print("Something went wrong")

finally:

 if f !=None:

  f.close()

 print("Program completely executed")
```

Q.) Explain about "with" statement.

- **"with" statement is an implicit resource relasing statement.**
- **"with" keyword of Python language is used to create the "with" statement.**

```
#filewith.py

filename=input("Which file to read and display:")

try:
```

```
 with open(filename,"r") as f:
  data=f.read()
  print(data)
except:
 print("File doesn't exist")
print("program successfully excuted")
```

Note:- If file is opened successfully, "with" statement automatically closes the file, programmer is disburdened of closing the file explicitly in finally block.

Q.) What are the properties of file object?

- **name:- gives the name of the file**
- **closed:- specifies whether file is closed or not**
- **mode:- gives the mode of the file**
- **readable():-returns True if the file is readable**
- **writable():-returns True if the file is writeable**

```
f=open("one.txt","r+")
print("Name of the file:",f.name)  #one.txt
print("mode of the file:",f.mode) # r+
print("Is it writable:",f.writable())#True
print("Is file readable:",f.readable()) #False
print("Is the file closed:",f.closed) #False
```

Q.) Explain about seek() and tell() methods of file object.

- **seek() moves the File Pointer to the specified location**
- **tell() returns the location of the file pointer.**

Q.) Explain about pickling and unpickling in the context of File Handling in Python?

- **The process of storing object(in fact, object's data) into a file is nothing but pickling and retrieving the object from disk file is nothing but unpickling.**

- pickling fecilitates Python object's persistence.

Q)Example progrm on pickling.

```
#pickleexample.py
import pickle
class Student:
 def __init__(self,rollno,name):
  self.rollno=rollno
  self.name=name
 def displayStudentDetails(self):
  print("ROLLNO:",self.rollno)
  print("NAME:",self.name)
s=Student(1001,"Rama")
f=open("student.dat","wb")
pickle.dump(s,f)
print("student object is persisted")
f.close()
```

Q.) Example program on unpickling

```
#unpickleexample.py
import pickle
class Student:
 def __init__(self,rollno,name):
  self.rollno=rollno
  self.name=name
 def displayStudentDetails(self):
  print("ROLLNO:",self.rollno)
  print("NAME:",self.name)
f=open("student.dat","rb")
```

```
s=pickle.load(f)

s.displayStudentDetails()

f.close()
```

08/07/2019

Q.) Explain about assertion

- **Assertion statement is nothing but an assertion in Python program.**
- **Using assert keyword of python language assert statement is written in Python program.**

Syntax:

1. **Assert <condition>**
2. **Assert <condition> <statement>**

- **If condition is evaluated to true asserted does not do anything in the program flow continue.**
- **If condition is false then pvm raise assertion error message given by programmer flow is halted.**
- **In second version assert statement an error message given by programmer is additionally printed functionality of the assertion statement is the same.**
- **Ascertain facilitates the programmer to debug their code during development phase.**
- **during production phase debugging should not be done there for assertion can be disabled.**
- **Exception handling should be done even for production environment also there for assertion or not alternative exception handling.**

Q.) Example of assertion program.

```
#assert.py

n=int(input("Enter a number: "))

assert n>0,"please do not enter –ve number"

print("program continue…")
```

Note:-assertion can be disabled as follows while executing the python program.

Py –o assert.py

Decorator

Q.) What is decorator?

- Decorator is a specialised function that enhance the power functionality of other function without changing that functions code.
- To which function a decorated is applied at function called decorated function.
- Decorator are used to add secondary logic like transaction management identification input data validation etc, to main business data processing function without disturbing them.

Q.) How to create a decorator?

1. Define another function that takes a function object reference as argument.
2. Define a nested function
3. In that state function write the secondary logic
4. Call business method using function reference
5. Return the nested function within the outer function..

Q.) What is the purpose of decorator

- Decorator separates business logic and secondary project dynamically.

Q.) write a python program in which, decorator is created and applied to a business function.

```
#decoratorexample.py

def manageTransaction(f): #decorator creation

 def addsecondary():

  print(“transaction begin”)

  f() #calling the business method

  print(“transaction ends”)

 return addsecondary #returning the nested function

@manageTransaction   #decoration applied
```

```
def process():
 print("business logic goes here")
process()    #primary logic function call
```

output:

transaction begin

business logic does here

transaction ends

Q.) What is decorator chaining

- Implied more than one decorator to a business function is known as decorator chaining.

```
@authenticate
@manageTransaction
def process():
 -------------
 -------------
```

Generator

Q.) explain about generators?

- Generator is a special function with at least one yield statement.
- Generator returns an iterator object to loop over.
- When a generator is called with function name, it's code is not executed instead, an iterator object is returned.
- When next() function is called by supplying the iteration object generator starts execution upto yield statement, when yield statement is encountered then function (generator function) is paused and control is returned to the caller.
- When yield statement is executed control comes out of the generator function is resume with next() execution start from the line after yield statement and so on.

- **Whenever large sets of data are there for processing loading all data items at a time into memory causes MemoryError using generator one data item at a time can be loaded and processed.**
- **Generator provide memory efficiency as no items are loaded into memory in advance.**

Q.) example program on generator

```
#generatorexample.py

def counter():

 print("generator execution starts……")

 count=0

 yield count

 print("generator execution again……")

 count=count+1

 yield count

 print(" generator execution last time…")

 count=count+1

 yield count
```

Q.) what is a generator expression?

- **A tuple comprehension (comprehension enclosed between parenthesis) is nothing but a generator expression.**

For eg.

```
G=(x for x in range (1,10))
```

- **Generator expression produces an iteration object for looping.**

```
print[next(G)]          #1

print[next(G)]          #2
```

Q.) what is the output of following code:

```
l=[x for x in range(1000000000000)]

print(l[0])
```

ans:-MemoryError

Q.) what is the output of following code:

```
l=[x for x in range(1000000000000)]
print(next(l))
```

ans:- 0

09/07/2019

Database Programming using Python

Q.) What is Database programming using Python?

- **A Python program performing CRUD operations with the database is nothing but database programming using Python.**

C- CREATION OF DATA(STORING DATA INTO THE DATABASE)

R-RETRIEVING DATA

U-UPDATING DATA

D-DELETING DATA

Q.) What are the steps involved in Python-database programming?

Step 1: Establish the connection with the database server

Step 2:- create the Cursor object

Step 3:- Submit the SQL Statement

Step 4:- End the transaction for INSERTION,DELETION and UPDATION

Step 5:- close the cursor object after performing database operations

Step 6:- close the connection object

Q.) What is the role of Cursor object in Python-database programming?

1. **It is used to submit SQL statement to the DBMS.**
2. **It holds the data received from the database.**

Q)Write a Python program that stores an employee data into the database.

```
#storeemployee.py

import mysql.connector

connection=mysql.connector.connect(host='localhost',passwd='root',user='root',database='empdb')

cursor=connection.cursor()

cursor.execute("INSERT INTO EMPLOYEE VALUES(1002,'Rahim',50000)")

connection.commit()

print("Number of employees updated:",cursor.rowcount)

connection.close()
```

Q.) Write a Python Program that updates a record in the database.

```
#updateemployee.py

import mysql.connector

connection=mysql.connector.connect(host='localhost',passwd='root',user='root',database='empdb')

cursor=connection.cursor()

sqlstatement="UPDATE EMPLOYEE SET SALARY=SALARY+1000"

cursor.execute(sqlstatement)

connection.commit()

print("Number of employees updated:",cursor.rowcount)

connection.close()
```

Q.) Write a Python program that deletes data in the database.

```
#deleteemployee.py

import mysql.connector

connection=mysql.connector.connect(host='localhost',passwd='root',user='root',database='empdb')

cursor=connection.cursor()
```

```
sqlstatement="DELETE FROM EMPLOYEE WHERE EMPNO=1001"

cursor.execute(sqlstatement)

connection.commit()

print("Number of employees deleted:",cursor.rowcount)

connection.close()
```

Q.) Write a Python Program that retrieves a record from the database.

```
#retrieveemployee.py

import mysql.connector

connection=mysql.connector.connect(host='localhost',passwd='root',user='root',database='empdb')

cursor=connection.cursor()

sqlstatement="SELECT * FROM EMPLOYEE WHERE EMPNO=1001"

cursor.execute(sqlstatement)

emprecord=cursor.fetchone()

print("Employee details...")

print("EMPNO:",emprecord[0])

print("NAME:",emprecord[1])

print("SALARY:",emprecord[2])

cursor.close()

connection.close()
```

Q.) Write a Python Program that retrieves multiple records from the database.

```
import mysql.connector

connection=mysql.connector.connect(host='localhost',passwd='root',user='root',database='empdb')

cursor=connection.cursor()

sqlstatement="SELECT * FROM EMPLOYEE"
```

```
cursor.execute(sqlstatement)

emprecords=cursor.fetchall()

for emprecord in emprecords:

 print(emprecord[0],emprecord[1],emprecord[2])

cursor.close()

connection.close()
```

Q.) Write a Python program that stores multiple employee records into the database by reading data from the keyboard.

```
#storeemployees.py

import mysql.connector

connection=mysql.connector.connect(host='localhost',user='root',pas
swd='root',database='empdb')

cursor=connection.cursor(prepared=True)

query="INSERT INTO EMPLOYEE VALUES(%s,%s,%s)"

for i in range(2):

 empno=int(input("Enter empno:"))

 name=input("Enter name:")

 salary=int(input("Enter salary:"))

 cursor.execute(query,(empno,name,salary))

 connection.commit()

 print(cursor.rowcount,"employee stored successfully")

cursor.close()

connection.close();
```

10/07/2019

Multithreading

Q.) What are the similarities between multitasking and multithreading?

- **Both are meant for performing multiple tasks concurrently**
- **Both are meant for optimum utilization of CPU**
- **OS support is required for both of them in terms of time slicing/sharing and context switching**

Q.) What are the differences between multitasking and multithreading?

- **process based multiple tasks performing mechansim is multitasking**
- **thread based(sub process) multiple tasks performing mechanism is multithreading**
- **No programming effort required for multitasking. OS implicitly provides support. Where as for multithreading, programming effort is required.**

Q)What is a thread?

- **A single sequential flow of control is nothing but a thread.**

OR

- **An independent path of execution is nothing but a thread**

Q)What is a single threaded application?

- **If an application has only one flow of control, it is known as a single threaded application.**

Q)What is the limitation of a single threaded application?

- **Multiple concurrent tasks can't be performed in a single threaded application.**

Q)What is a multithreaded application?

- **An application is said to be multithreaded if it has multiple flows of control.**
- **A multithreaded application facilitates multiple concurrent tasks in a single application.**

Q)What is multithreading?

- **The process of making an application a multithreaded application is nothing but multithreading.**

Q)How to make an application multithreaded?

Step 1:- Write a user defined class that inherits from Thread class

Step 2:- Override(implement) run() method in the user defined class. Place task performing code directly/indirectly in run method

Note:- run() is the body of the thread.

Step 3:- Create as many user defined thread objects as there are concurrent flows in the application.

Step 4:- activate the threads by calling start() on the user defined thread object.

Note:- When start() method is called on thread object, their body is attached to them and they become active and are in ready queue expecting CPU cycles.

Q.) Develop a multithreaded Python application in which two concurrent flows to be created.

1. **in one flow print numbers from 1 to 20**
2. **in another flow print numbers from 21 to 40**

```
#multithread1.py

from threading import *

import time

class MyThread1(Thread):

 def run(self):

  for i in range(1,21):

   print("My thread 1:",i)

   time.sleep(1)


class MyThread2(Thread):

 def run(self):

  for i in range(21,41):

   print("My thread 2:",i)

   time.sleep(1)
```

```
t1=MyThread1()

t2=MyThread2()

t1.start()

t2.start()
```

Q.) Modify the above application so as to have only one user defined thread class.

```
#multithread2.py

from threading import *

import time

class MyThread(Thread):

 def run(self):

  if self.getName()=="t1":

   for i in range(1,21):

    print("My thread 1:",i)

    time.sleep(1)

  else:

   for i in range(21,41):

    print("My thread 2:",i)

    time.sleep(1)


t1=MyThread()

t2=MyThread()

t1.setName("t1")

t2.setName("t2")

t1.start()

t2.start()
```

Q.) Modify the first multithreaded application so as to place task performing code in another method and invoke it in the body of the thread.

```
#multithread3.py

from threading import *

import time

class A:

  def displayOneToTwenty(self):

   for i in range(1,21):

    print(i)

    time.sleep(1)

  def displayTwentyOneToForty(self):

   for i in range(21,41):

    print(i)

    time.sleep(1)

class MyThread(Thread):

  def run(self):

   if self.getName()=="t1":

    a.displayOneToTwenty()

   else:

    a.displayTwentyOneToForty()


a=A()

t1=MyThread()

t2=MyThread()

t1.setName("t1")

t2.setName("t2")
```

```
t1.start()

t2.start()
```

Q.) Write a POP style of Python program to implement multithreading.

```
#popmultithread.py

from threading import *

import time

def displayOneToTwenty():

   for i in range(1,21):

    print(i)

    time.sleep(1)

def displayTwentyOneToForty():

   for i in range(21,41):

    print(i)

    time.sleep(1)

t1=Thread(target=displayOneToTwenty)

t2=Thread(target=displayTwentyOneToForty)

t1.start()

t2.start()
```

Q.) What is synchronization?

- **Whenever multiple threads are acting on the same shared resource, there is a chance of data inconsistency.**
- **Synchronization prevents such inconsistency and makes the multithreaded applications "thread safe".**
- **Synchronization is the mechanism of allowing only one thread at a time to access a shared resource whenever multiple threads are acting on a shared resource.**

Q)Write a Python Program to prove that without synchronization, data inconsistency occurs when multiple threads act upon a shared resource.

```
#bankingapp.py

from threading import *

import time

class Account:

 def __init__(self,accno,balance):

  self.accno=accno

  self.balance=balance

 def getBalance(self):

  return self.balance

 def withdraw(self,wamount):

  print(current_thread().getName(),"Trying to withdraw
from",self.accno)

  print(current_thread().getName(),"Getting balance")

  balance=self.getBalance()

  print(current_thread().getName()," got Balance is Rs.",balance)

  if balance>wamount:


   print(current_thread().getName(),"withdrawn Rs.",wamount)

   self.balance=self.balance-wamount

   print(current_thread().getName()," updated balance to
Rs.",self.balance)

  else:

   print("Insufficient funds for withdrwal for
",current_thread().getName())

class AccountThread(Thread):
```

```
 def run(self):
  acc.withdraw(wamount)


acc=Account(100001,50000)
wamount=30000
t1=AccountThread()
t2=AccountThread()
t1.start()
t2.start()
t1.join()
t2.join()
print("Final balance in the account is Rs.",acc.getBalance())
```

Q.) What is the purpose of join() method?

- In which flow that method is called on a thread object, that thread is made inactive. After that thread completes its execution(dead) then only the current flow(thread) becomes active.

Q.) Modify the above application so as to make it thread safe.

```
#bankingapp2.py
from threading import *
import time
class Account:
 def __init__(self,accno,balance):
  self.accno=accno
  self.balance=balance
 def getBalance(self):
  return self.balance
 def withdraw(self,wamount):
```

```
  print(current_thread().getName(),"Trying to withdraw from
",self.accno)

  print(current_thread().getName(),"Getting balance")

  balance=self.getBalance()

  print(current_thread().getName()," got Balance is Rs.",balance)

  if balance>wamount:


   print(current_thread().getName(),"withdrawn Rs.",wamount)

   self.balance=self.balance-wamount

   print(current_thread().getName()," updated balance to
Rs.",self.balance)

  else:

   print("Insufficient funds for withdrwal for
",current_thread().getName())

class AccountThread(Thread):

 lock=Lock()

 def run(self):

  self.lock.acquire()

  acc.withdraw(wamount)

  self.lock.release()


acc=Account(100001,50000)

wamount=30000

t1=AccountThread()

t2=AccountThread()

t1.start()

t2.start()

t1.join()
```

```
t2.join()

print("Final balance in the account is Rs.",acc.getBalance())
```

Regular Expressions

Q.) What is a Regular Expression?

- A regular expression is a special text that describes a search pattern.
- Regular Expressions are useful in extracting information from the given data.

Q.) What are the meta characters used in regular expressions?

- Meta characters are characters that are interpreted in a special way

  [] . ^ $ * + ? {} () \ |

- [] a set of characters can be specified using [] for matching.
- . matches any single character except new line(\n)
- ^ starts with
- $ matches if a string ends with a certain character
- matches zero or more occurences of its preceding character.
- + used to match one or more occurences of its preceding character.
- ? used to match zero or one occurence of its preceding character.
- {} used to match the specified number of repetetions
- () used to group sub patterns
- \ used to escape characters
- | match for either this or that character(or operator)

Q.) What are the quantifiers?

- A quantifier specifies the number of occurences to match.

1.)a #exactly one 'a'

2.)a+ #Atleast one 'a'

3.)a* #Zero or more 'a's.

4.)a? #Zero or One 'a'

5.)a{n} #Exactly 'n' number of a's

Q.) What are the character classes?

1.)[abc] #either a OR b OR c

2.)[^abc] #Except a and b and c

3.)[a-z] #Any lower case alphabet

4.)[A-Z] #Any upper case alphabet

5.)[a-zA-Z] #Any alphabet

6.)[0-9] # Any digit between 0 and 9

7.)[a-zA-Z0-9] #Any alphanumerical character

8.)[^a-zA-Z0-9] #special characters other than alphanumerical

Q.) What are the pre-defined character classes?

1.)\s #Space character

2.)\S #Any character except space

3.)\d #Any digits 0-9

4.)\D #Any character except digit

5.)\w #Any word character

6.)\W #Any character other than word character.

7.) . #Any character including special character.

Q.) How does Python support for regular expressions in Python applications?

- **A built-in module "re" is given to support regular expressions in Python.**
- **"re" module has provided the following functions**
- **match():-checks the given pattern at the beginning of the string. If match is found, this function returns Match object.Otherwise, None**

#matchexample.py

import re

```
datastring="xyzxylmn"

searchstring=input("Enter the search string:")

m=re.match(searchstring,datastring)

if m!=None:

 print("specified string is found in data")

else:

 print("Specified string is not found")
```

- fullmatch():-Used to match the pattern to the whole data string. If matched it returns Match object otherwise None.

```
#validmobileno.py

import re

mobilenumber=input("Enter the mobile number:")

m=re.fullmatch("[6-9]\d{9}",mobilenumber)

if m !=None:

 print(mobilenumber,"is valid")

else:

 print("It is not a valid mobile number")
```

- findall():- This function is used to find all the occurrences of the match. It returns a list that contains all the occurrences.

```
#mailid.py

import re

s="This is abc123@gmail.com   to xyz@gmail.com and lmn@gmail for urgent reply"

list=re.findall("\S+@\S+[\.]\S+",s)

for mailid in list:

 print(mailid)

 #vehiclereg.py
```

```
import re

vehicleno=input("Enter the vehicle registration number:")

m=re.fullmatch("TS\s[012][0-9]\s[A-Z]{2}\s\d{4}",vehicleno)

if m!=None:

 print("Valid registration number")

else:

 print("In valid registration number")
```

- sub():- This function is used to substitute/replace the matched pattern in the data string with the specified content.

```
#subexample.py

import re

data="That language is the best. that language is used every where"

newdata=re.sub("[T|t]\S{3}","Python",data)

print(newdata)
```

- search():- This function is used to search in the data string with the specified pattern. If match is found it returns Match object. Otherwise None.

```
#searhexample.py

import re

datastring="Learning Python is very easy"

result=re.search("[e]\S{3}$",datastring)

if result !=None:

 print("Ends with easy")

else:

 print("Doesn't end with easy")
```

Logging

Q.) Explain about logging in a Python application?

- **The process of keeping track of the events that occur during the execution of a python Application and recording them is nothing but logging.**
- **Logging is important for software development, debugging and running.**
- **Logging information in your application is critical to the understanding of the runtime behaviour of any application, especially in cases when you encounter unexpected scenarios, errors, or just for tracking certain activities.**
- **3 entities(objects) involve in the implementation of logging in a Python application.**

1. **logger**
2. **handler**
3. **formatter**

- **Handler objects are responsible for dispatching the log messages to the specified destination. For example, to the console, file or an email**

- **Formatters specify the layout of the resultant log record.**
- **Logger object does 3 things.**

1. **provides methods to application developers to perform logging.**
2. **determines which log messages to log(record) based on log level specified.**
3. **Passes on the log messages to all the configured handlers.**

- **We have two kinds of loggers.**

1. **root logger(default logger)**
2. **custom loggers**

- **To perform logging mostly custom loggers are used as they are more flexible.**

Q.) What are different log levels?

- **Log levels are simply a means of categorizing the entries in your log file.**
- **DEBUG:- These are used to give Detailed information, typically of interest only when diagnosing problems.**
- **INFO:- These are used to Confirm that things are working as expected**

- **WARNING:- These are used an indication that something unexpected happened, or indicative of some problem in the near future.**
- **ERROR:- This tells that due to a more serious problem, the software has not been able to perform some function**
- **CRITICAL:- This tells serious error, indicating that the program itself may be unable to continue running.**

Note:- Whenever a log level is set, messages of that level and levels higher than that only are logged.

Q.) How to implement logging in a Python application?

Step 1:- import the "logging"module

Step 2:- create the logger object.

Step 3:- set the log level

Step 4:- create handlers as per requirement

step 5:- set the levels for handlers

Step 6:- create formatters

Step 7:- set the formatters to handlers

Step 8:- add the handlers to logger

Step 9:- perform logging

Q.) Example program on logging

```
#logging1.py

import logging

logger=logging.getLogger(__name__)

logger.setLevel(logging.DEBUG)

fileHandler=logging.FileHandler('log1.txt',mode='w')

streamHandler=logging.StreamHandler()

fileHandler.setLevel(logging.INFO)

formatter=logging.Formatter('%(asctime)s:%(levelname)s:%(message)s'
,datefmt='%d/%m/%y %H:%M:%S')
```

formatter2=logging.Formatter('%(asctime)s:%(message)s',datefmt='%d/%m/%y %H:%M:%S')

fileHandler.setFormatter(formatter)

streamHandler.setFormatter(formatter2)

logger.addHandler(fileHandler)

logger.addHandler(streamHandler)

logger.debug("debugging info")

logger.info("working info")

logger.warning("unexpected happend")

logger.error("problem.please rectify")

logger.critical("program can't run")

Q.) How to implement logging using an external logging configuration file.

Step 1:- create a configuration file(text file with any name and any extension)

Step 2:- import the necessary modules

Step 3:- load the configuration file

Step 4:- create the logger object

Step 5:- perform logging

Q.) Modify the previous application so as to configure logging information into a file.

Note:- There are two files for this program.

1. logging.conf
2. logging2.py

#logging.conf

[loggers]

keys=root,custom

```
[handlers]
keys=streamHandler,fileHandler

[formatters]
keys=formatter1,formatter2

[logger_root]
handlers=streamHandler

[logger_custom]
level=DEBUG
handlers=fileHandler
qualname=custom

[handler_fileHandler]
class=FileHandler
level=WARNING
formatter=formatter1
args=('log.txt','w')

[handler_streamHandler]
class=StreamHandler
formatter=formatter2
level=CRITICAL

[formatter_formatter1]
format=%(asctime)s:%(levelname)s:%(message)s
```

```
datefmt=%d/%m/%y %H:%M:%S

[formatter_formatter2]
format=%(asctime)s:%(message)s
datefmt=%d/%m/%y %H:%M:%S
```

```
#logging2.py
import logging
import logging.config
logging.config.fileConfig('logging.conf')
logger=logging.getLogger("custom")
logger.debug("debugging info")
logger.info("working info")
logger.warning("unexpected happend")
logger.error("problem.please rectify")
logger.critical("program can't run")
```
